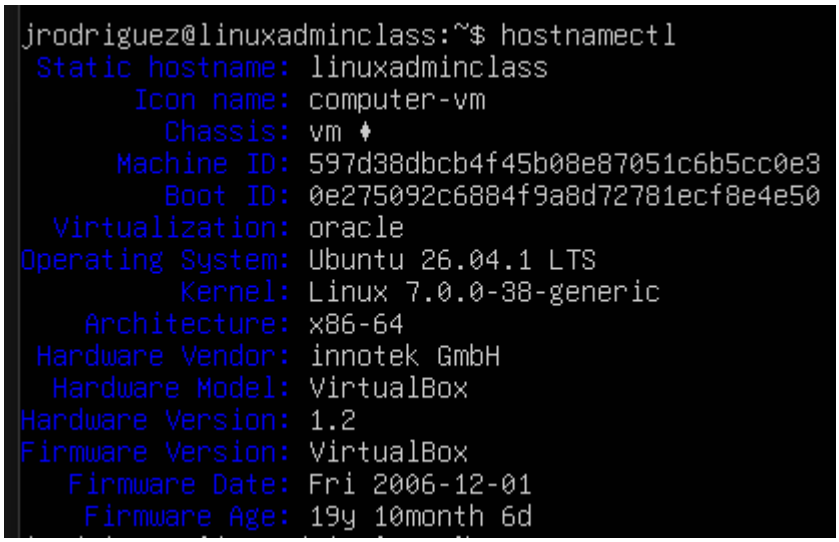
Joel Rodriguez
CIS-245
Fall 2026
Documentation

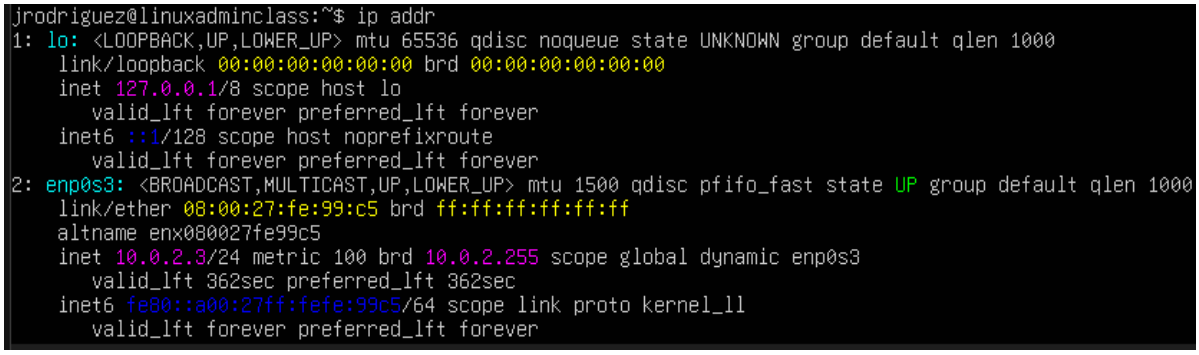
UBUNTU

1. Identify the Server
    a. Run: *hostnamectl*



This command provides system info including the hostname, OS, Kernel version, and system architecture. Recording this info helps identify which server the network config belongs to and provides useful system info for trouble shooting.

    b. View Network Interfaces and IP Addresses: *ip addr*



The command shows the network interfaces and IP addresses configured on the server. The lo interfaceis the loopback interface

and uses the address 127.0.0.1 for communication within the local system. The primary network interface is enp0s3, which is currently up and assigned to the IP of 10.0.2.3.

c. View routing info and default gateway: *ip route*

```
jrodriguez@linuxadminclass:~$ ip route
default via 10.0.2.1 dev enp0s3 proto dhcp src 10.0.2.3 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.3 metric 100
10.0.2.1 dev enp0s3 proto dhcp scope link src 10.0.2.3 metric 100
172.19.40.5 via 10.0.2.1 dev enp0s3 proto dhcp src 10.0.2.3 metric 100
172.19.40.11 via 10.0.2.1 dev enp0s3 proto dhcp src 10.0.2.3 metric 100
172.19.40.12 via 10.0.2.1 dev enp0s3 proto dhcp src 10.0.2.3 metric 100
```

This command displays the server's routing table and identifies the path network traffic takes to reach other networks. This server has a default gateway of 10.0.2.1 through the enp0s3 interface. The server's local network is 10.0.2.0/24 and its source ip is 10.0.2.3. This also shows the routing info was obtained through DHCP.

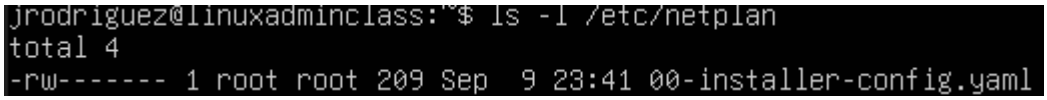
d. View DNS config: *resolvectl status*

```
jrodriguez@linuxadminclass:~$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 2 (enp0s3)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 172.19.40.11
       DNS Servers: 172.19.40.5 172.19.40.11 172.19.40.12
     Default Route: yes
```

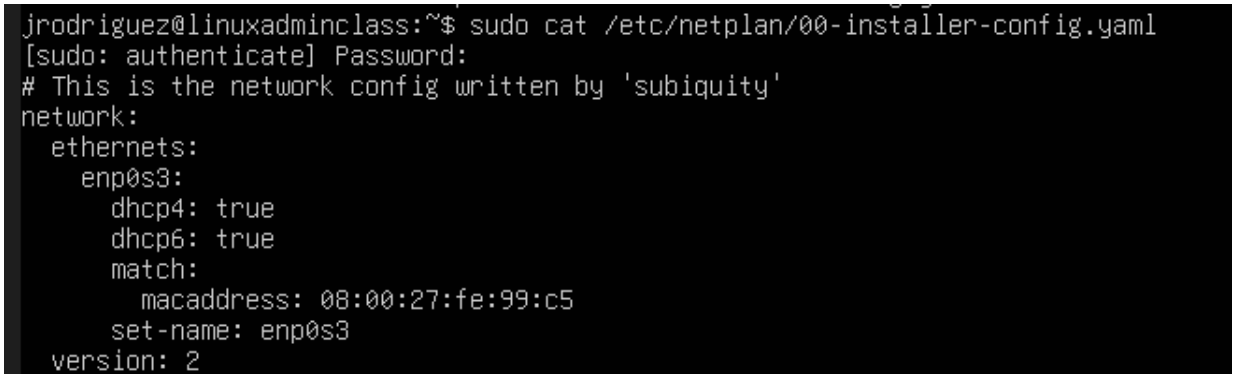
This status command shows the DNS config being used the server. The enp0s3 interface is configured to use the DNS servers 172.19.40.5, 172.19.40.11(listed as current DNS server), and 178.19.40.12.

e. Locate the network config files: *la -l /etc/netplan*



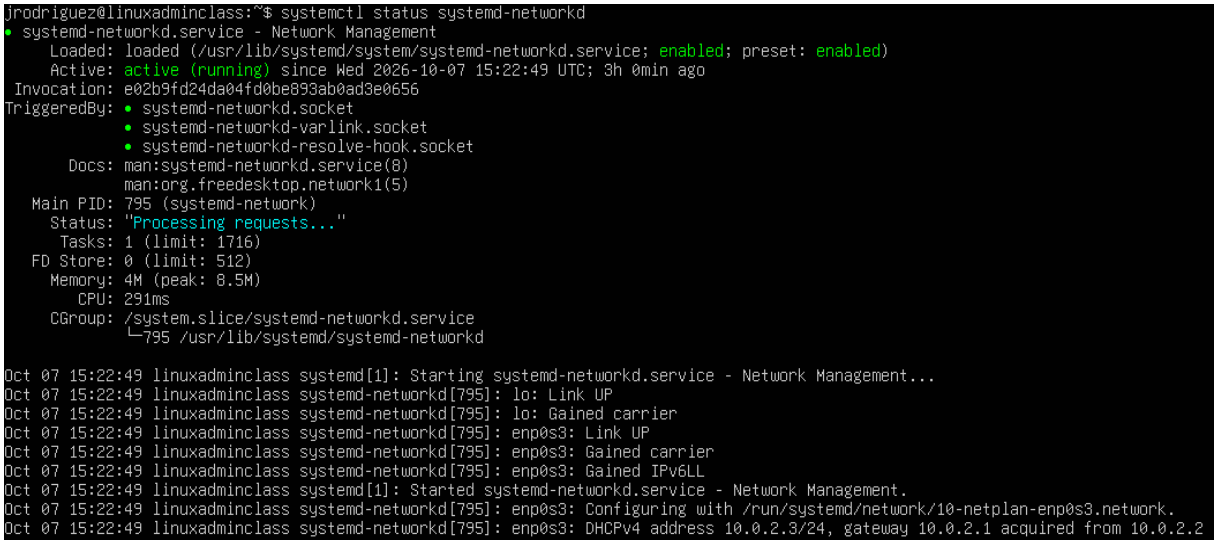
This command identified where the network config files are stored in the netplan directory. Output shows 00-installer-config.yaml is the name of the file which is owned by the root user.

f. View the network config file: *sudo cat /etc/netplan/00-installer-config.yaml*



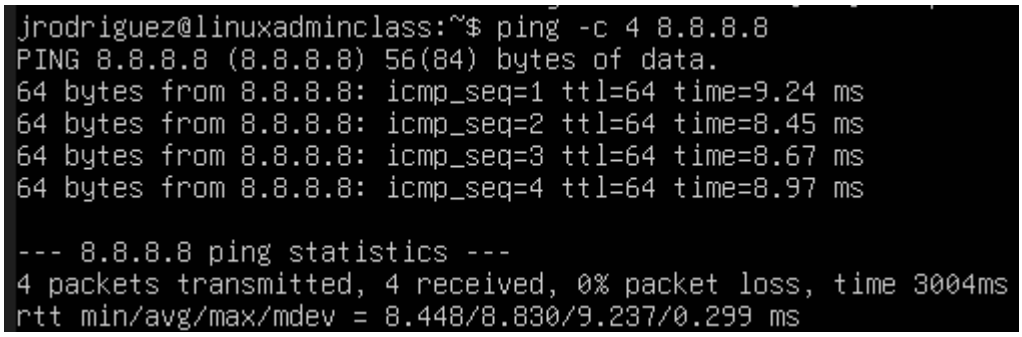
This viewed the contents of the server's netplan config file. The file configures the enp0s3 Ethernet interface and shows "dhcp4:" and "dhcp6:" as true, which mean the DHCP is enabled for IPv4 and IPv6. The file also identifies the network adapter by its MAC address.

g. Check the network management service: *systemctl status system-networkd*



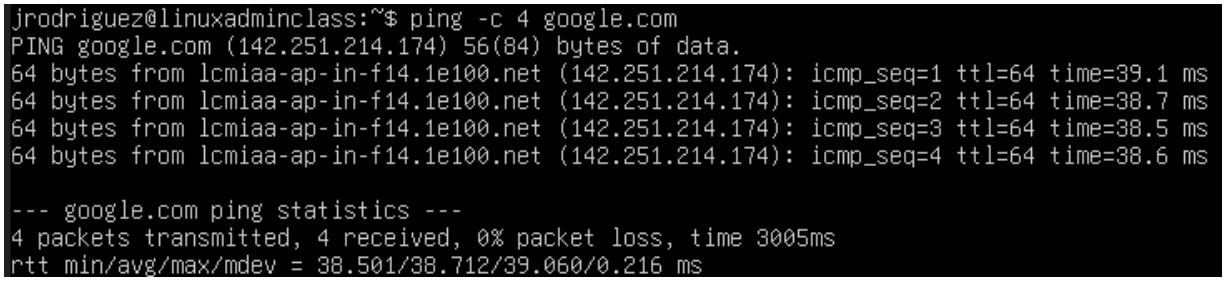
Output shows that system-networkd is enabled and active. It also confirms the interface is up and obtained the IPv4 address and the default gateway through DHCP.

h. Test the network connectivity: *ping -c 4 8.8.8.8*



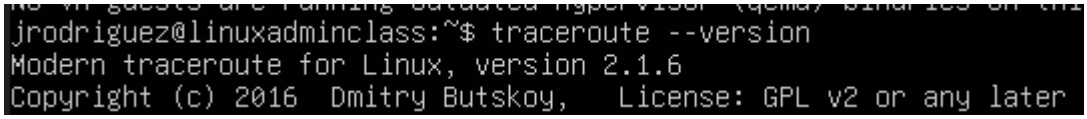
This confirms that the server has working network connectivity outside of its local network. All four packets were successfully returned with 0% packet loss.

i. Test DNS name resolution: *ping -c 4 google.com*



This ping test required the server to resolve with the domain name google.com to an IP address beofre communicating with it. All four packets were successfully returned with 0% packet loss.

j. Trace the network path: *traceroute google.com*
   Command was not found
   i. We're going to install traceroute:
      sudo apt -o Acquire::ForceIPv4=true install traceroute
      Program Installed: TraceRoute
      Date Installed: 10/7/2026
   ii. Enter password to authenticate
   iii. Verify the install: *traceroute –version*

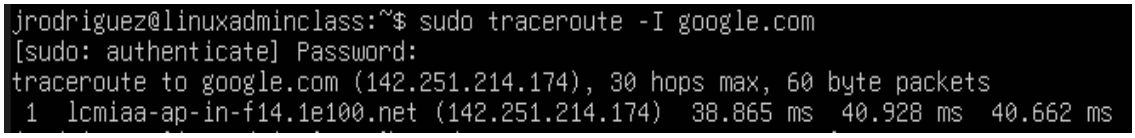


iv. Trace the path: *traceroute google.com*



This command traces the network path from the server towards google's network. The first hop successfully reached my default gateway, but the remaining hops displayed * * *, meaning no responses were received from those devices. Not sure why this is an issue because I was able to successfully ping it.

v. If "*traceroute google.com*" doesn't work, trace the path using ICMP: *sudo traceroute -I google.com*



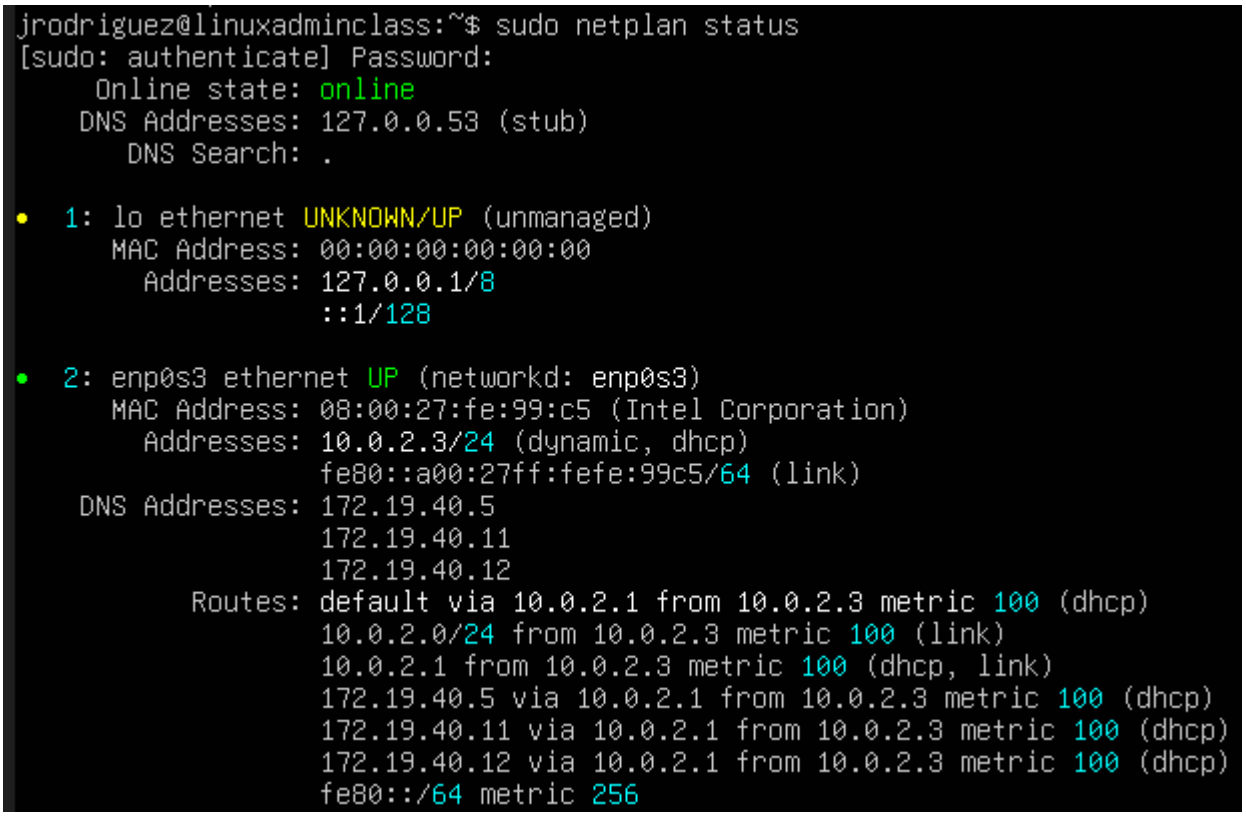
Since the previous traceroute didn't reach google, I ran this command to test the connection using ICMP. This command successfully reached google's IP. The destination appeared on the first hop. This confirms that my server can reach Google.

2. **Explain how the networking works by default**

My ubuntu server uses Netplan to configure its network settings and systemd-networkd to manage network connections. The main config file is located at /etc/netplan/00-installer-config.yaml. The file identifies my network interface as enp0s3 and shows that both DHCPv4 and DHCPv6 are enabled. DHCP allows the server to automatically receive networksettings instead of requiring me to configure everything manually. My server

received the IPv4 addres 10.0.2.3/24 and uses 10.0.2.1 as its default gatwat. DNS information is managed through systemd-resolved, which allows the server to translate domain names into IP addresses. Since this server runs in Virtualbox its vitual network adapter provides connectivity through the configured virtual network.

We'll verify the network management tool: *sudo netplan status*



This command verified the current network configuration on my server. The results showed that the server was onlineand that the enp0s3 network interface was active and managed by system-networkd. The interface received the IPv4 address 10.0.2.3/24 through DHCP and uses 10.0.2.1 as its default gateway. The command also displayed the configured DNS servers and routing information. This is useful because it allows a network admin to review the server's network settings and confirm that the interface is configured correctly.

a. Locate the DNS config file: *ls -l /etc/resolv.conf*



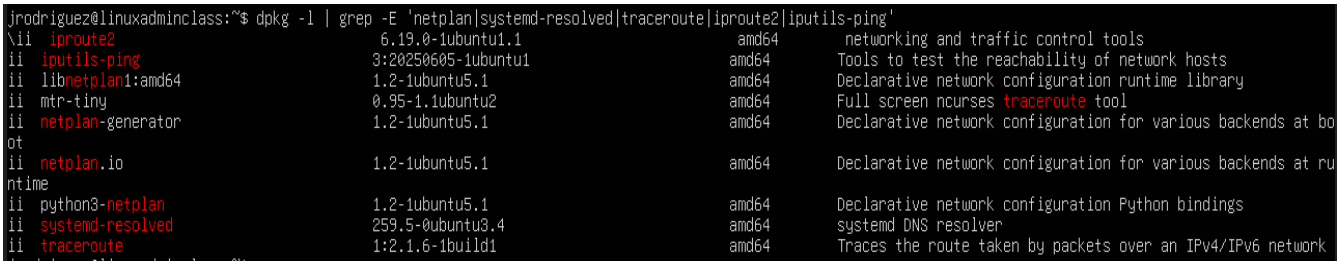
This command located the DNS resolver configuration file the server. The results showed that etc/resolv.conf is a symbolic link pointing to /run/systemd/resolve/stub-resolv.conf. This means ubuntu uses a configuration file generated by systemd-resolved to handle DNS requests. This is important because it helps admins understand where the DNS settings are managed and which files to check when troubleshooting name resolution problems.

Important Ubuntu Network config files

File	Purpose
/etc/resolv.conf	Standard location applications use to find DNS resolver settings
/run/systemd/resolve/stub-resolv.conf	Generated DNS configuration file managed by systemd-resolved
/etc/netplan/00-installer-config.yaml	Defines the server's network interface and DHCP configuration

3. Installed Networking Programs and Updates
    a. Identify installed Networking Programs

dpkg -l | grep -E 'netplan | systemd-resolved | traceroute | iproute2 | iptutils-ping'



This command identifies the network programs installed on the server and check their package versions. The results show the tools such as netplan, iproute2, iputils-ping, systemd-resolved, and traceroute are installed. These programs help manage network interfaces, configure routing, resolve domain names, and troubleshoot connectivity problems.

    b. Check installation and update dates

*sudo zgrep -h -E 'Install: | Upgrade:' /var/log/apt/history.log**

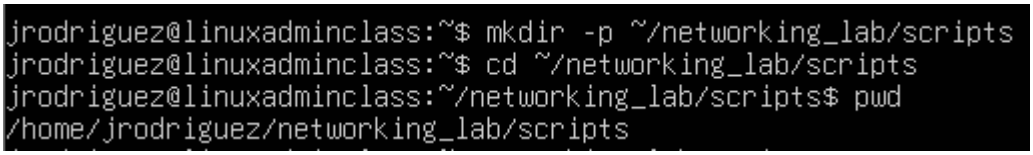
*sudo zgrep -h -B 3 -A 2 'traceroute:amd64' /var/log/apt/history.log**

I checked the server's APT history logs to review software installations and updates. The results showed records of programs installed on the server, including networking and system admin tools such as fail2ba, cowsay, lolcat, and traceroute. I also checked the detailed installation history for traceroute confirmed that it was installed on oct 7, 2026 at 7:50PM using the command sudo apt -o Acquire::ForceIPv4=true install traceroute. I used this command because the server was having trouble connecting to ubuntu's package repos, and forcing IPv4 allowed the installation to complete successfully. Traceroute is useful for troubleshooting network connectivity because it helps identify the path packets take to reach a destination. Reviewing these installations logs helps admins keep

track of software changes, installation dates and commands used to maintain the server.

4. Network information Dump Script
    a. Create a directory for scripts

i. Enter: *mkdir -p ~/networking_lab/scripts*
ii. Then: *cd ~/networking_lab/scripts*
iii. Verify location: pwd `



b. Create the bach script: *nano network_info_dump.sh*

c. Time to enter the script

```
#!/bin/bash

# Ubuntu Networking Lab
# Purpose: Display network information and save it to a file.

#Create a filename containing my name, date, and purpose
DATE=$(date +%Y-%m-%d)
OUTPUT="$HOME/networking_lab/rodriguez_${DATE}_network_info.txt"

#Displays information and save it to the ouput file
{
        echo "================================================"
        echo "               NETWORK INFORMATION              "
        echo "================================================"

        echo
        echo "Date: $(date)"

        echo
        echo "===== SERVER INFORMATION ====="
        hostnamectl

        echo
        echo "===== NETWORK INTERFACES ====="
        ip addr

        echo
        echo "===== ROUTING INFORMATION ====="
        ip route

        echo
        echo "===== DNS CONFIGURATION ====="
        resolvectl status

        echo
        echo "===== NETWORK SERVICE STATUS ====="
        systemctl is-active systemd-networkd

        echo
        echo "===== NETWORK CONNECTIVITY ====="
        ping -c 4 google.com

        echo
        echo "===== NETWORK TRACE (ICMP) ====="
        traceroute -I -m 10 -w 1 google.com
}  2>&1 | tee "$OUTPUT"
```

d. Save and exit Nano:
    i. Press Ctrl+O to save

    ii. Enter to confirm
    iii. Crtl+X to exit
e. Make the script executable: *chmod +x network_info_dump.sh*
f. Verify: *ls -l network_info_dump.sh*
g. Run the script: *./network_info_dump.sh*
   I created a Bash script called network_info_dump.sh to automatically collect important networking information from my server. The script displays server information, network interfaces, routing information, DNS configuration, network service status, and connectivity test results. I also configured the script to use ICMP traceroute because it worked more successfully than the default UDP traceroute during my earlier testing. After running the script, I confirmed that the ping test successfully transmitted and received four packets with 0% packet loss and that ICMP traceroute reached Google's server. The script also saved the results to a text file containing my last name, the date, and the purpose of the report. The script would be useful for a network administrator because it collects several important troubleshooting details automatically instead of requiring each command to be entered separately.
h. Verify the saved report
    i. Run: *ls -lh ~/networking_lab/*network_info.txt*
    ii. Then: *cat ~/networking_lab/Rodriguez_2026-10-11_network_info.txt*

5. Interactive Network Troubleshooting script
    a. Open a new shell script: *nano network_troubleshooter.sh*

b. Add a bash script

```
#!/bin/bash

# Ubuntu Networking Troubleshooter
# Purpose: Purpose: Help troubleshoot ubuntu and CentOS networking.
# This script uses variables, conditionals and a loop.

echo "========================================="
echo "       LINUX NETWORK TROUBLESHOOTER      "
echo "========================================="

# Ask the user to select a Linux server.
echo "Which Linux Server are you using?"
echo "1. Ubuntu"
echo "2. CentOS"
read distro

# Check the user's selection.
if [ "$distro" = "1" ]
then
        echo "You selected Ubuntu."
elif [ "$distro" = "2" ]
then
        echo "You selected CentOS."
else
        echo "Invalid server selection."
        exit 1
fi

# Repeat the menu until the user chooses to exit.
choice=0

while [ "$choice" != "6" ]
do
        echo
        echo "===== NETWORK TROUBLESHOOTING MENU ====="
        echo "1. No Internet connection"
        echo "2. Configure a static IP address"
        echo "3. Check DNS settings"
        echo "4. Run traceroute"
        echo "5. Check network interfaces"
        echo "6. Exit"
        echo "Enter your choice:"
        read choice

        case "$choice" in
```

```
1)
        echo "Check your IP address:"
        echo "ip addr"
        echo "Check your default gateway:"
        echo "ip route"
        echo "Test your Internet Connection:"
        echo "ping -c 4 8.8.8.8"

        if [ "$distro" = "1" ]
        then
                echo "Check Ubuntu network services:"
                echo "systemctl status systemd-networkd"
        else
                echo "Check CentOS network service:"
                echo "systemctl status NetworkManager:"
        fi
        ;;

2)
        echo "STATIC IP CONFIGURATION"

        if [ "$distro" = "1" ]
        then
                echo "Ubuntu uses Netplan."
                echo "Find your configuration file:"
                echo "ls /etc/netplan"
                echo "Edit the correct YAML file:"
                echo "sudo nano /etc/netplan/00-installer-config.yaml"
                echo "After editing, test the configuration:"
                echo "sudo netplan try"
                echo "Check your IP address:"
                echo "ip addr"
        else
                echo "CentOS uses NetworkManager."
                echo "View your connections:"
                echo "nmcli connection show"
                echo "Use nmcli to configure a static IP."
                echo "Check your IP address:"
                echo "ip addr"
        fi
        ;;
```

```
                ;;
        3)
                echo "DNS TROUBLESHOOTING"
                if [ "$distro" = "1" ]
                then
                        echo "Check Ubuntu DNS settings:"
                        echo "resolvectl status"
                else
                        echo "Check CentOS DNS settings:"
                        echo "nmcli device show"
                fi

                echo "View the DNS resolver file:"
                echo "cat /etc/resolv,conf"
                echo "Test domain name resolution:"
                echo "ping -c 4 google.com"
                ;;
        4)
                echo "Enter a website to trace:"
                read website

                if [ -z "$website" ]
                then
                        echo "No website was entered."
                else
                        echo "Run this ICMP traceroute command:"
                        echo "traceroute -I $website"
                fi
                ;;
        5)
                echo "NETWORK INTERFACE INFORMATION"
                echo "View IP addresses:"
                echo "ip addr"
                echo "View routing information:"
                echo "ip route"
                ;;
        6)
                echo "Exiting the troubleshooting script."
                ;;
        *)
                echo "Invalid option. Try again."
                ;;
        esac
done
```

c. Save and exit Nano:
    i. Press Ctrl+O to save
    ii. Enter to confirm
    iii. Crtl+X to exit
d. Make the script executable: *chmod +x network_troubleshooter.sh*
e. Verify: *ls -l network_troubleshooter.sh*

f. Test it: *./network_troubleshooter.sh*

```
jrodriguez@linuxadminclass:~/networking_lab/scripts$ ./network_troubleshooter.sh
========================================
       LINUX NETWORK TROUBLESHOOTER
========================================
Which Linux Server are you using?
1. Ubuntu
2. CentOS
1
You selected Ubuntu.

===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
```

```
===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
1
Check your IP address:
ip addr
Check your default gateway:
ip route
Test your Internet Connection:
ping -c 4 8.8.8.8
Check Ubuntu network services:
systemctl status systemd-networkd

===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
2
STATIC IP CONFIGURATION
Ubuntu uses Netplan.
Find your configuration file:
ls /etc/netplan
Edit the correct YAML file:
sudo nano /etc/netplan/00-installer-config.yaml
After editing, test the configuration:
sudo netplan try
Check your IP address:
ip addr
```

```
Enter your choice:
3
DNS TROUBLESHOOTING
Check Ubuntu DNS settings:
resolvectl status
View the DNS resolver file:
cat /etc/resolv,conf
Test domain name resolution:
ping -c 4 google.com

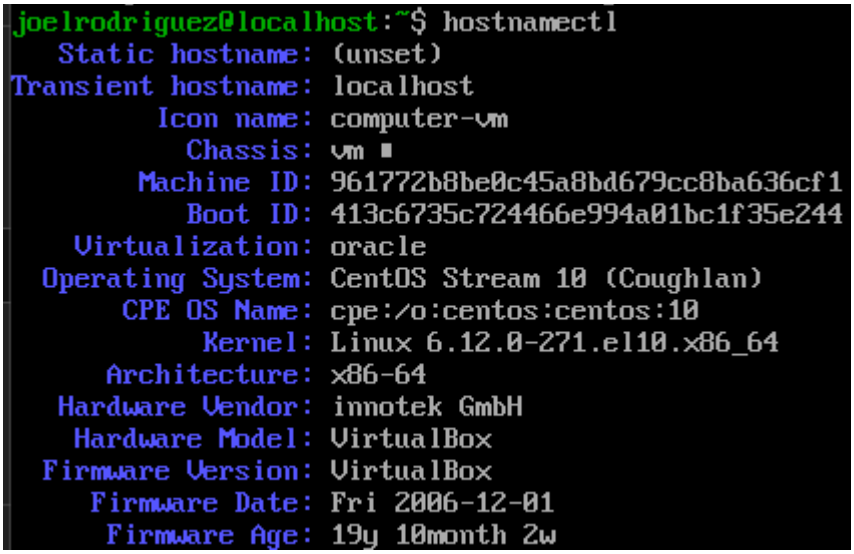
===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
4
Enter a website to trace:
5
Run this ICMP traceroute command:
traceroute -I 5

===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
```

I created a bash srcipt called network_troubleshooter.sh to provide an interactive troubleshooting guide for ubuntu and CentOS Linux servers. The script uses basic shell scripting consepts, including variables, read statements, if and else conditions, a case statement, and while loop. When the script starts, it asks the user which Linux distribution they're troubleshooting and then displays a menu of common networking problems. These include internet connectivity issues, static IP configuratios, DNS troublehsooting, traceroute testing, and checking network interfaces. Depending on the OS selected, the script provides the appropriate commands and troubleshooting suggestions. After correcting a few syntax errors, I successfully ran the script and verified that the interactive menu worked. The script would be helpful to a network admin because it provides organized troubleshooting instuctions and allows multiple issues to be reviewed without restarting the program.

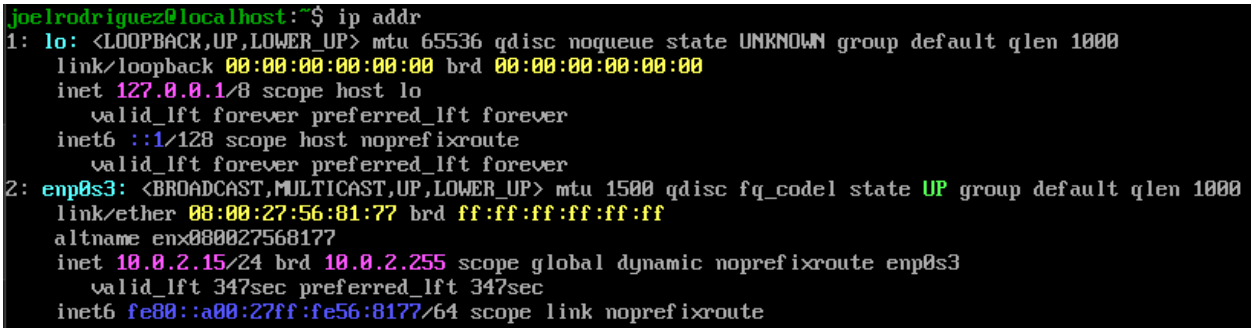
CENTOS

1. Identify the server: *hostnamectl*



I used the hostnamectl command to display info about my CentOS Server, where it includes its hostname, OS, and kernel version. The results confirmed that my VM is running CentOS Stream 10 on Virtualbox. I also noticed that the static hostname was not configured and the system was using localhost.

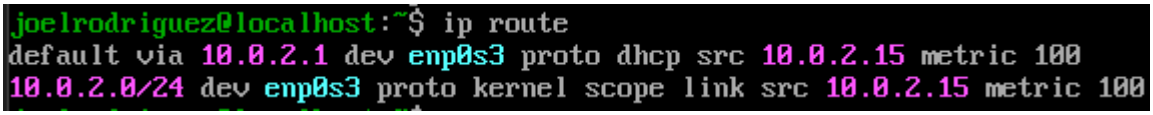
2. Checking IP address and network interface: *ip addr*



I used the ip addr command to identify IP addresses and the network

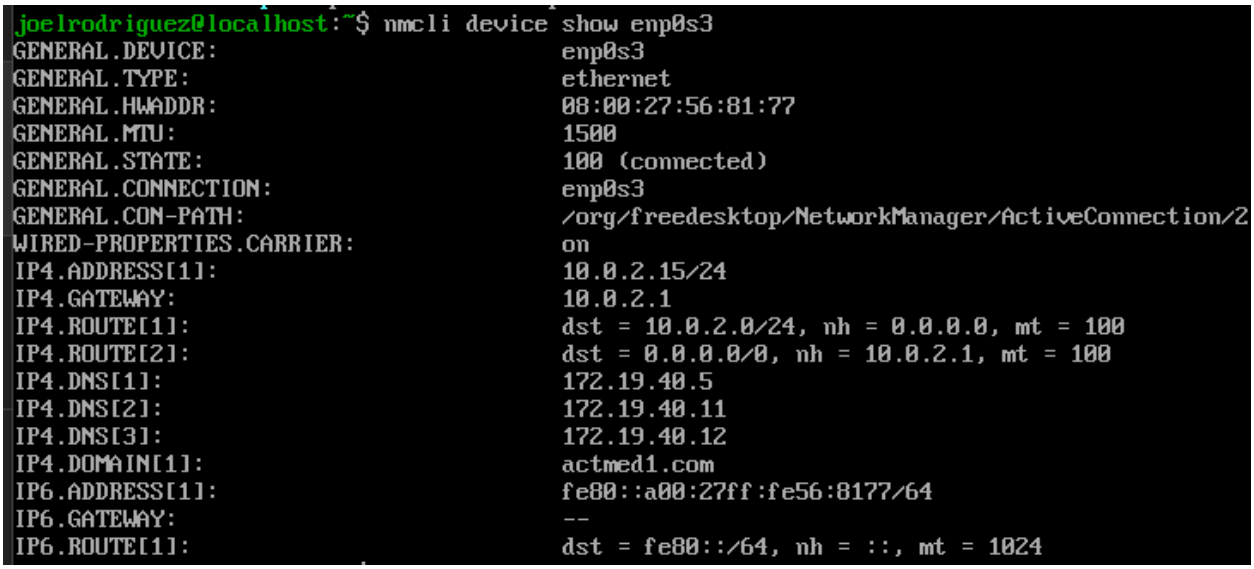
interfaces assigned to the CentOS server. You can see that the enp0s3 was active and had an IP address of 10.0.2.15/24, assigned through DHCP. This command helped me understand how to verify whether a network interface is enabled and properly configured.

3. Checking the routing table: *ip route*



I used the ip route command to view the routing table on my CentOS server. The results showed a default gateway of 10.0.2.1 through the enp0s3 interface. This helped me understand how the server determines where to send the network traffic when communicating with devices outside its local network.

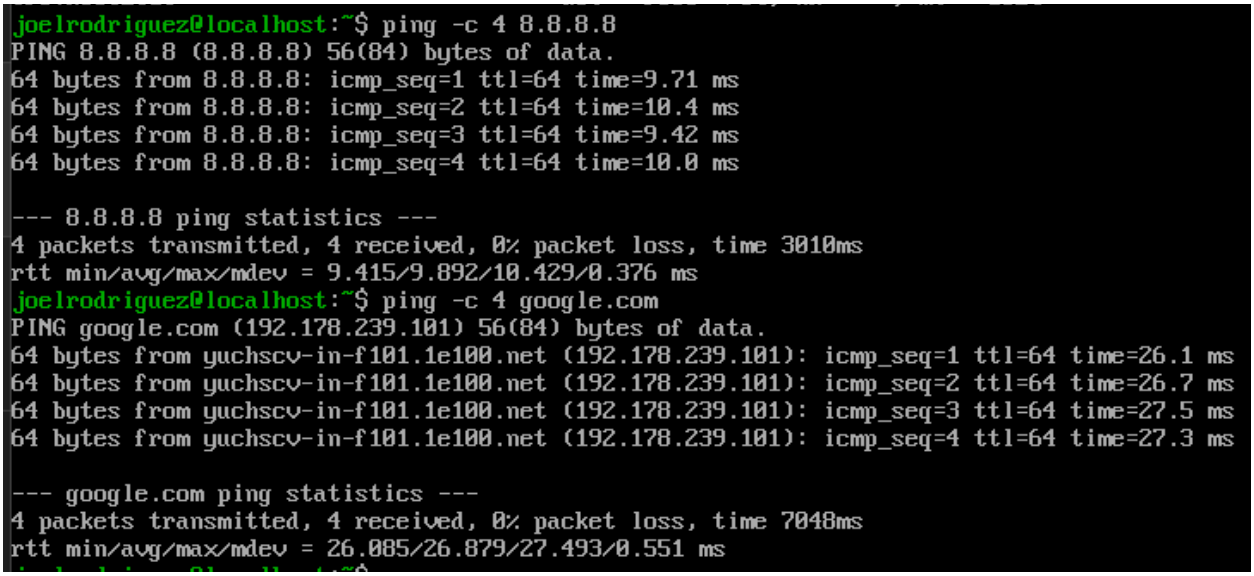
4. Checking DNS config: *nmcli device show enp0s3*



I used the nmcli device show enp0s3 command to examine my centOS server's network configuration, including its DNS settings. The output

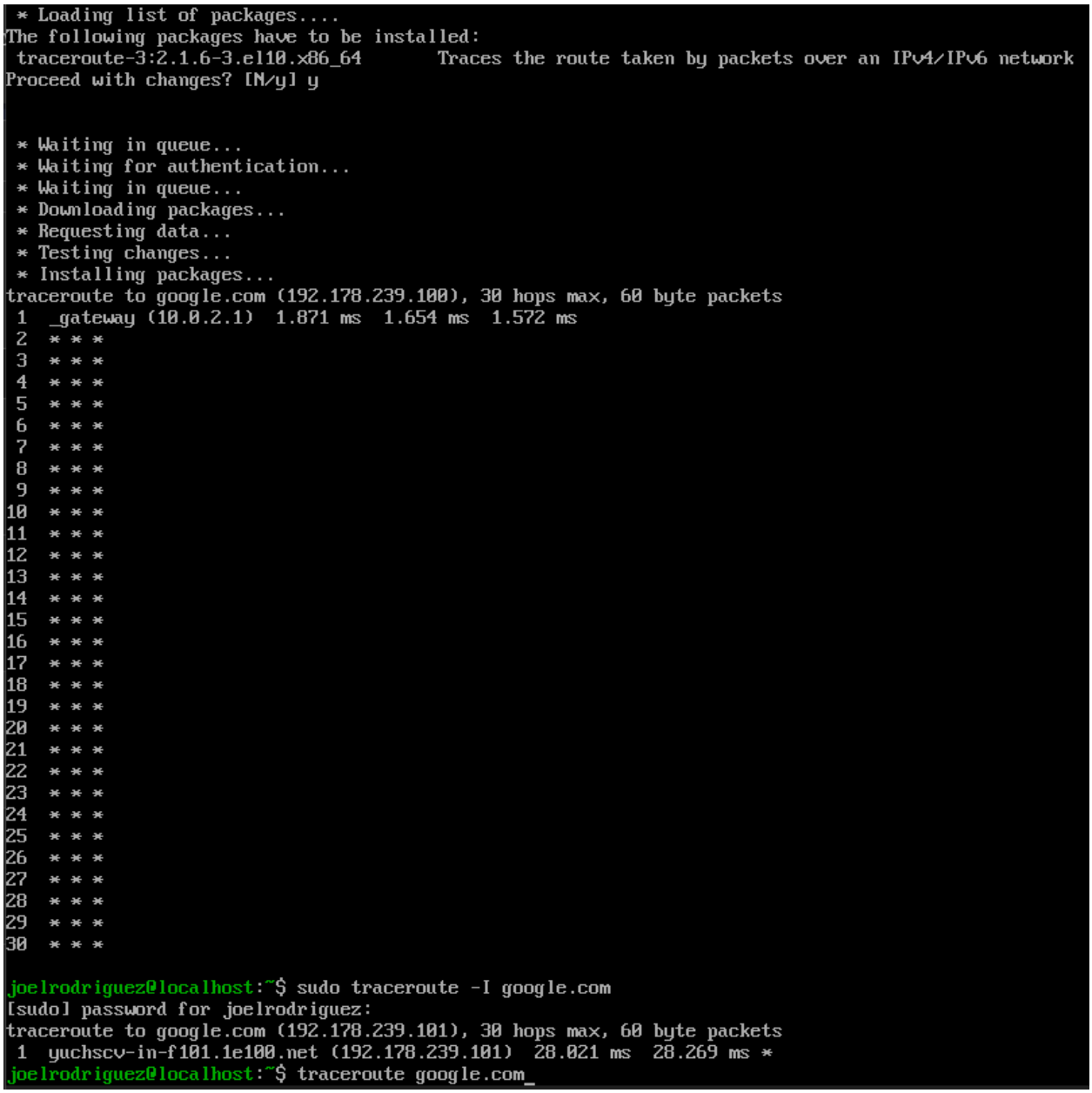
showed three configured DNS servers and confirmed that the network interface was connected.

5. Testing network connectivity: *ping -c 4 8.8.8.8* and *ping -c 4 google.com*



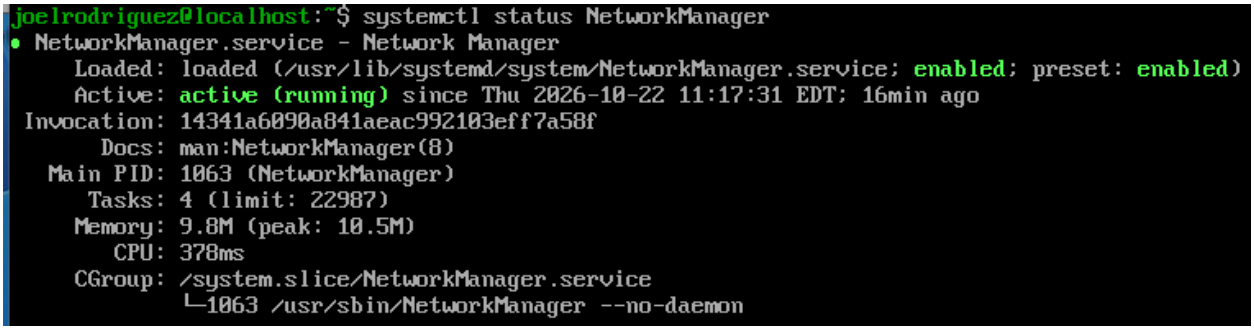
I used the ping command to test network connectivity on the server. First, I pinged the 8.8.8.8 to verify that the server could communicate with an external IP address, then I pinged google.com to test the connectivity and DNS name resolution. Both tests were successful with 0% packet loss, confirming that the server could reach external networks and resolve domain names.

6. Install traceroute into the Server: *sudo dnf install traceroute -y*
    a. Test network route: *traceroute google.com*
    b. If the command fails, run: *sudo traceroute -I google.com*



I used the traceroute command to examine the network path from my CentOS server to google.com. The first attempt identified my gateway at 10.0.2.1, but the remaining hops timed out. I then tried *sudo traceroute -I google.com*, which forces it to use ICMP, and it successfully reached the Google's IP address on the first hop.

7. Checking the NetworkManager Service: *systemctl status NetworkManager*



I used this command to check the status of the Network Manager service on my server. The output showed that the service was enabled and actively running. This confirmed the NetworkManager was operating properly and managing the server's network connections.

8. Checking Network connections: *nmcli connection show*



This command viewed the network connections configured on the server. Output displayed two connections: enp0s3 and lo. We identified the available network connections and devices associated with them.

9. Network interface status: *nmcli device status*

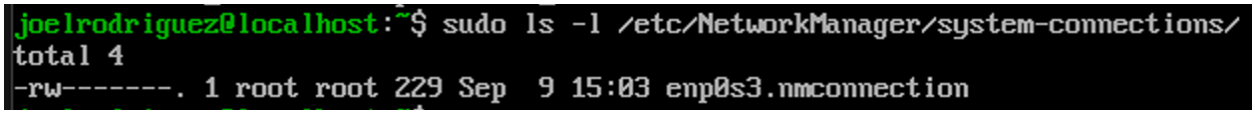


This command checked the status of the server's netowrk interfaces. Shows the ethernet interface enp0s3 was connected and that the loopback interface lo was also connected.

10. Explain how Networking works by default

    My CentOS Stream 10 server uses NetworkManager to manage its network connections and settings. The main network interface is enp0s3, which receives its IP address automatically through DHCP. My server was assigned the IP address 10.0.2.15/24 and uses 10.0.2.1 as its default gateway. DNS servers are also provided through the network configuration, allowing the server to resolve domain names into IP addresses. Since my server is running in VirtualBox, it uses a virtual network adapter to communicate with other networks.

    a. Locate the NetworkManager configuration file*:*
    sudo ls -l /etc/NetworkManager/system-connections/

    I used the `sudo ls -l /etc/NetworkManager/system-connections/` command to locate the network configuration file on my CentOS server. The output showed a file named `enp0s3.nmconnection`, which stores the connection settings managed by NetworkManager. I also noticed that the file is owned by root and has restricted permissions, which helps protect the network configuration from unauthorized changes.

    b. View the configuration settings:
    sudo cat /etc/NetworkManager/system-connections/enp0s3.nmconnection

```
joelrodriguez@localhost:~$ sudo cat /etc/NetworkManager/system-connections/enp0s3.nmconnection
[sudo] password for joelrodriguez:
[connection]
id=enp0s3
uuid=3b00232f-8b24-384c-b28d-5cf3afef799a
type=ethernet
autoconnect-priority=-999
interface-name=enp0s3
timestamp=1788979298

[ethernet]

[ipv4]
method=auto

[ipv6]
addr-gen-mode=eui64
method=auto

[proxy]
```

I used the `sudo cat /etc/NetworkManager/system-connections/enp0s3.nmconnection` command to view the network configuration file on my CentOS server. The output showed that the connection uses the enp0s3 Ethernet interface and that IPv4 is set to automatic, allowing the server to receive its IP address through DHCP. IPv6 is also configured automatically. This helped me understand how NetworkManager stores and manages the network settings for my server.

c. Verify the NetworkManager status: *nmcli general status*

```
joelrodriguez@localhost:~$ nmcli general status
STATE      CONNECTIVITY  WIFI-HW  WIFI     WWAN-HW  WWAN     METERED
connected  full          missing  enabled  missing  enabled  no (guessed)
```

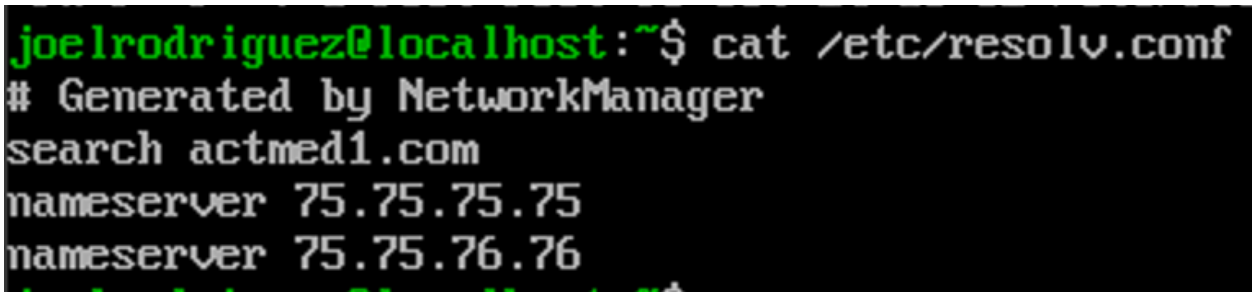
I used the `nmcli general status` command to check the overall network status of my CentOS server. The output showed that NetworkManager was connected and reported full network connectivity. It also showed that no Wi-Fi hardware was detected, which makes sense because my server uses a virtual Ethernet adapter in VirtualBox. This command helps me quickly verify that the server has an active network connection.

d. Locate the DNS configuration file: *ls -l /etc/resolv.conf*

```
joelrodriguez@localhost:~$ ls -l /etc/resolv.conf
-rw-r--r--. 1 root root 95 Oct 24 16:52 /etc/resolv.conf
```

I used the `ls -l /etc/resolv.conf` command to locate the DNS resolver configuration file on my CentOS server. The output showed that `/etc/resolv.conf` is a regular file owned by root, rather than a symbolic link like the one on my Ubuntu server. This file is used to provide DNS resolver settings, which help the server translate domain names into IP addresses when accessing websites or other network resources.

e. View the DNS resolver settings: *cat /etc/resolv.conf*



I used the `cat /etc/resolv.conf` command to view the DNS configuration on my CentOS server. The output showed that the file was generated by NetworkManager and listed two DNS servers, 75.75.75.75 and 75.75.76.76, along with the search domain actmed1.com. This helped me identify which DNS settings my server was using and where to check them if I needed to troubleshoot domain name resolution.

File	Purpose
/etc/NetworkManager/system-connections/enp0s3.nmconnection	Stores the Ethernet connection profile, including automatic IP configuration
/etc/resolv.conf	Contains the DNS resolver settings generated by NetworkManager
/etc/NetworkManager/	Main directory for NetworkManager

configuration files

11. Installed networking programs and updates
    a. Identify installed programs:
    rpm -q NetworkManager iproute iputils traceroute

```
joelrodriguez@localhost:~$ rpm -q NetworkManager iproute iputils traceroute
NetworkManager-1.58.0-1.el10.x86_64
iproute-7.0.0-2.el10.x86_64
iputils-20240905-5.el10.x86_64
traceroute-2.1.6-3.el10.x86_64
```

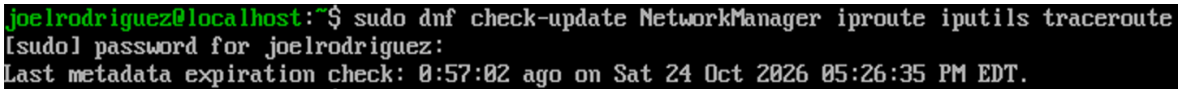
    I used the `rpm -q` command to identify the networking programs installed on my CentOS server and check their versions. The output confirmed that NetworkManager, iproute, iputils, and traceroute were installed. These programs are useful for managing network connections, checking IP addresses and routing information, testing connectivity, and tracing network paths.

    b. Check Installation and Update Dates:
    rpm -q --last NetworkManager iproute iputils traceroute

```
joelrodriguez@localhost:~$ rpm -q --last NetworkManager iproute iputils traceroute
traceroute-2.1.6-3.el10.x86_64                Thu 15 Oct 2026 03:22:16 PM EDT
iputils-20240905-5.el10.x86_64                Wed 09 Sep 2026 02:54:14 PM EDT
NetworkManager-1.58.0-1.el10.x86_64           Wed 09 Sep 2026 02:52:34 PM EDT
iproute-7.0.0-2.el10.x86_64                   Wed 09 Sep 2026 02:52:31 PM EDT
```

I used the `rpm -q --last` command to check when the networking packages were installed on my CentOS server. The output showed that NetworkManager, iproute, and iputils were installed on September 9, 2026, while traceroute was installed on October 15, 2026. This command helps me keep track of when networking programs were installed or updated on my server.

c. Check Available Package Updates:
sudo dnf check-update NetworkManager iproute iputils traceroute



I used the `sudo dnf check-update` command to check whether updates were available for NetworkManager, iproute, iputils, and traceroute on my CentOS server. The command completed without listing any available updates, indicating that the installed versions were up to date according to the repository information. This command is useful for checking networking software updates without installing or changing anything on the server.

12. Testing the network info script
    a. List the directory*: ls -l ~/networking_lab/scripts/*
    b. Run the script: *~/networking_lab/scripts/network_info_dump.sh*

```
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 460sec preferred_lft 460sec
    inet6 fe80::a00:27ff:fe56:8177/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

===== ROUTING INFORMATION =====
default via 10.0.2.1 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100

===== DNS CONFIGURATION =====
GENERAL.DEVICE:                         enp0s3
GENERAL.TYPE:                           ethernet
GENERAL.HWADDR:                         08:00:27:56:81:77
GENERAL.MTU:                            1500
GENERAL.STATE:                          100 (connected)
GENERAL.CONNECTION:                     enp0s3
GENERAL.CON-PATH:                       /org/freedesktop/NetworkManager/ActiveConnection/2
WIRED-PROPERTIES.CARRIER:               on
IP4.ADDRESS[1]:                         10.0.2.15/24
IP4.GATEWAY:                            10.0.2.1
IP4.ROUTE[1]:                           dst = 10.0.2.0/24, nh = 0.0.0.0, mt = 100
IP4.ROUTE[2]:                           dst = 0.0.0.0/0, nh = 10.0.2.1, mt = 100
IP4.DNS[1]:                             172.19.40.5
IP4.DNS[2]:                             172.19.40.11
IP4.DNS[3]:                             172.19.40.12
IP4.DOMAIN[1]:                          actmed1.com
IP6.ADDRESS[1]:                         fe80::a00:27ff:fe56:8177/64
IP6.GATEWAY:                            --
IP6.ROUTE[1]:                           dst = fe80::/64, nh = ::, mt = 1024

===== NETWORK SERVICE STATUS =====
active

===== NETWORK CONNECTIVITY =====
PING google.com (173.194.46.101) 56(84) bytes of data.
64 bytes from yucmhdr-in-f101.1e100.net (173.194.46.101): icmp_seq=1 ttl=64 time=26.1 ms
64 bytes from yucmhdr-in-f101.1e100.net (173.194.46.101): icmp_seq=2 ttl=64 time=24.8 ms
64 bytes from yucmhdr-in-f101.1e100.net (173.194.46.101): icmp_seq=3 ttl=64 time=24.2 ms
64 bytes from yucmhdr-in-f101.1e100.net (173.194.46.101): icmp_seq=4 ttl=64 time=24.3 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 7047ms
rtt min/avg/max/mdev = 24.189/24.833/26.090/0.762 ms

===== NETWORK TRACE (ICMP) =====
traceroute to google.com (173.194.46.139), 30 hops max, 60 byte packets
 1  yucmhdr-in-f139.1e100.net (173.194.46.139)  24.842 ms  24.600 ms *

Report saved to /home/joelrodriguez/networking_lab/rodriguez_2026-10-22_network_info.txt
joelrodriguez@localhost:~/networking_lab/scripts$
```

I modified the network_info_dump.sh script that I created in my ubuntu server so it can be tailored to the CentOS server. I updated the network service command to check NetworkManager instead of system-networkd. After running the script, it displayed my server information, network interfaces, routing table, DNS config, and network service status. It also performed ping test and an ICMP traceroute to google.com. The script collected network information and reported the location of the saved text file, allowing me to review the results later without running each command individually.

13. Test the interactive network troubleshooter:
    ~/network_lab/scripts/network_troubleshooter.sh

```
       LINUX NETWORK TROUBLESHOOTER
=========================================
Which Linux Server are you using?
1. Ubuntu
2. CentOS
2
You selected CentOS.

===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
3
DNS TROUBLESHOOTING
Check CentOS DNS settings:
nmcli device show
View the DNS resolver file:
cat /etc/resolv,conf
Test domain name resolution:
ping -c 4 google.com

===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
4
Enter a website to trace:
google.com
Run this ICMP traceroute command:
traceroute -I google.com

===== NETWORK TROUBLESHOOTING MENU =====
1. No Internet connection
2. Configure a static IP address
3. Check DNS settings
4. Run traceroute
5. Check network interfaces
6. Exit
Enter your choice:
6
Exiting the troubleshooting script.
```

I used the same network troubleshooter script from the ubuntu server and ran it on my Centos server. After Selecting the CentOS, the script displayed several options for diagnosing network problems, including internet connectivity, static IP configuration, DNS settings, traceroute, and network interfaces. I tested the DNS option, which displayed commands for checking DNS configuration, and the traceroute option, which allowed me to enter a website to trace. The script returned to main menu after each selection and exited successfully when I chose the exit option.

Research Paragraph

For my networking information dump script, I chose to include commands that would help me identify and troubleshoot common network problems on my Ubuntu and CentOS servers. After watching the Linux networking video, I learned how important it is to check things like IP addresses, network interfaces, DNS settings, and network services when troubleshooting. The video also explained that the ip command is a more modern alternative to ifconfig, which supports my decision to use ip addr and ip route in my script. I included hostnamectl to identify the server, ping to test connectivity, and traceroute to help identify the path network traffic takes to reach its destination. I think these additions are valuable because they allow me to collect important networking information in one place instead of running every command separately. Saving the results to a text file also makes it easier to review the server's network configuration and compare the results if problems occur in the future.

Cite: CIS 245 Week 5 Networking youtube video
https://www.youtube.com/watch?v=Za-9yTnMPG4

Running the Networking Scripts

I created two Bash scripts for this networking lab: a network information dump script and an interactive network troubleshooter. Both scripts are stored in the `~/networking_lab/scripts/` directory and can be run from the terminal using Bash. The network information dump script collects the server's hostname, operating system information, IP addresses, routing information, DNS settings, network service status, and connectivity test results. I chose these commands because they provide useful information for identifying and troubleshooting networking problems. The script also saves the results to a dated text file in the `~/networking_lab/` directory so I can review the information later.

The interactive network troubleshooter displays a menu where the user selects Ubuntu or CentOS and chooses a networking problem to investigate. The script provides suggested commands for checking internet connectivity, configuring a static IP address, troubleshooting DNS, running traceroute, and checking network interfaces. I included these options because they cover common networking issues that an administrator might encounter. Unlike the information dump script, the troubleshooter provides instructions rather than automatically making changes to the server.